News Topic Classifier + RAG Assistant

For my final project, I wanted to combine two techniques we learned in this class: finetuning a pretrained model and using Retrieval-Augmented Generation (RAG). I built a small “news assistant” that can take a question about current events, figure out what topic it belongs to (World, Sports, Business, or Sci/Tech), pull similar news articles, and then generate a short answer using the retrieved text.

I used the AG News dataset for both training the classifier and for the retrieval part. Since it already contains thousands of labeled news articles, it let me put everything together without having to build my own dataset.

1. Setup and library installation

In this section, I install the main Python libraries I need for the project. These include the Transformers library for BERT and T5, the Datasets library for loading AG News, SentenceTransformers for creating embeddings, and FAISS for building the retrieval index.

I also turn off logging tools like Weights & Biases so the training process doesn’t try to connect to any external services.

```
!pip install -q transformers datasets accelerate sentence-transformers faiss-cp
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 49.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 9.0 MB/s eta 0:00:00
```

I disable Weights & Biases logging so the Trainer doesn't try to connect to an external service.

```
import os
os.environ["WANDB_DISABLED"] = "true"
```

2. Loading the AG News dataset

Here I load the AG News dataset from Hugging Face. This dataset is commonly used for news topic classification and includes four categories: World, Sports, Business, and

Sci/Tech.

I use this dataset in two ways:

1. To train a BERT classifier that predicts the topic of a question or article.
2. As the collection of articles that my RAG system retrieves from.

After loading the dataset, I also grab the label names so it's easier to interpret the model's predictions later.

```
from datasets import load_dataset

ag_news = load_dataset("ag_news")
ag_news
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Userw
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access publ
  warnings.warn(
README.md:        8.07k/?  [00:00<00:00,  298kB/s]
data/train-00000-of-                                    18.6M/18.6M  [00:01<00:00,  32.5MB/s]
00001.parquet:  100%
data/test-00000-of-                                     1.23M/1.23M  [00:00<00:00,  3.00MB/s]
00001.parquet:  100%
Generating  train  split:  100%                 120000/120000  [00:00<00:00,  380004.11  examples/s]
Generating  test  split:  100%                      7600/7600  [00:00<00:00,  194611.08  examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 7600
```

```
label_names = ag_news["train"].features["label"].names
label_names
```

```
['World', 'Sports', 'Business', 'Sci/Tech']
```

3. Preprocessing and tokenization for BERT

Before training BERT, I need to tokenize the text. In this step, I load the `bert-base-uncased` tokenizer and combine each article's title and description into a single text field. I also apply truncation and padding so every input has the same length (128 tokens).

I then split the dataset into training and validation sets and convert each split into PyTorch format, which is required by the Hugging Face Trainer.

```
from transformers import AutoTokenizer

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

def preprocess(example):
    text = example["text"] # Changed from example["title"] + " " + example["des
    return tokenizer(text, truncation=True, padding="max_length", max_length=12

# Train/validation split
train_valid = ag_news["train"].train_test_split(test_size=0.1, seed=42)
train_ds = train_valid["train"]
valid_ds = train_valid["test"]
test_ds  = ag_news["test"]

train_ds = train_ds.map(preprocess, batched=True)
valid_ds = valid_ds.map(preprocess, batched=True)
test_ds  = test_ds.map(preprocess, batched=True)

# Set format for PyTorch
cols = ["input_ids", "attention_mask", "label"]
train_ds.set_format(type="torch", columns=cols)
valid_ds.set_format(type="torch", columns=cols)
test_ds.set_format(type="torch", columns=cols)
```

```
config.json: 100%                                   570/570 [00:00<00:00, 52.9kB/s]

tokenizer_config.json: 100%                         48.0/48.0 [00:00<00:00, 4.53kB/s]

vocab.txt:        232k/? [00:00<00:00, 5.50MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated req

tokenizer.json:        466k/? [00:00<00:00, 3.54MB/s]

Map: 100%                                   108000/108000 [00:27<00:00, 4187.50 examples/s]

Map: 100%                                     12000/12000 [00:04<00:00, 2678.18 examples/s]

Map: 100%                                      7600/7600 [00:01<00:00, 4729.15 examples/s]
```

4. Finetuning BERT for news topic classification

In this section, I set up a BERT model to classify news articles into the four AG News topics. The model has four output labels, one for each topic.

I also define two evaluation metrics:

- Accuracy: how often the model predicts the correct topic.
- Macro F1: the average F1 score across all four topics, so each one counts equally.

Next, I create the training arguments, which control things like the learning rate, batch size, number of epochs, and when to run evaluation. Since I'm using free Colab, I train on a smaller subset of the dataset (about 20k examples) to keep the training time reasonable.

```
import torch
from transformers import AutoModelForSequenceClassification, TrainingArguments,
import numpy as np
from evaluate import load

num_labels = 4
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=num_labels
)

accuracy_metric = load("accuracy")
f1_metric = load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_metric.compute(predictions=preds, references=labels)["accura
    f1_macro = f1_metric.compute(predictions=preds, references=labels, average=
    return {"accuracy": acc, "f1_macro": f1_macro}

training_args = TrainingArguments(
    output_dir="./bert-agnews",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=2,
    weight_decay=0.01,
    load_best_model_at_end=True,
    logging_steps=50,
    report_to="none",
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=valid_ds,
    compute_metrics=compute_metrics,
)
```

```
Loading weights: 100%   199/199 [00:00<00:00, 653.70it/s, Materializing param=bert.pooler.dense.we

BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                          | Status     |
---------------------------------------------+------------+-
cls.predictions.transform.dense.bias         | UNEXPECTED |
cls.predictions.bias                         | UNEXPECTED |
cls.seq_relationship.weight                  | UNEXPECTED |
cls.seq_relationship.bias                    | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |
cls.predictions.transform.dense.weight       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |
classifier.bias                              | MISSING    |
classifier.weight                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; r
- MISSING   :those params were newly initialized because missing from the checkp
```

5. Training the classifier

Here I use the Hugging Face Trainer API to finetune BERT on the AG News subset.

While the model trains, the notebook prints out things like:

- training loss
- validation loss
- validation accuracy and macro F1

These metrics help show how well the model is learning and whether it’s starting to overfit.

```
trainer.train()
```

[13500/13500 1:27:59, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	F1 Macro
1	0.180112	0.179186	0.944250	0.943932
2	0.134650	0.192586	0.949000	0.948728

```
Writing model shards: 100%                                    1/1 [00:07<00:00,  7.43s/it]

Writing model shards: 100%                                    1/1 [00:11<00:00, 11.75s/it]

There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerN
There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.Lay
TrainOutput(global_step=13500, training_loss=0.17748898748115258, metrics=
{'train_runtime': 5280.0592, 'train_samples_per_second': 40.909,
```

6. Evaluating on the held-out test set

After training, I evaluate the model on the official AG News test set. This set wasn't used during training, so it gives a good sense of how well the classifier generalizes.

The main metrics I look at are:

- test accuracy
- test macro F1

These scores show how well the model handles new, unseen news articles.

```
test_results = trainer.evaluate(test_ds)
test_results
```

[238/238 00:51]

```
{'eval_loss': 1.3996180295944214,
 'eval_model_preparation_time': 0.0029,
 'eval_accuracy': 0.2505263157894737,
 'eval_f1_macro': 0.1400269795653144,
 'eval_runtime': 52.3729,
 'eval_samples_per_second': 145.113,
 'eval_steps_per_second': 4.544}
```

7. Helper function for topic prediction

To make the classifier easier to use in the final application, I define a simple `classify_text` function. It takes any piece of text (like a user question), tokenizes it, and runs it through the finetuned BERT model.

The function returns:

- the predicted topic label (for example, “Sports”), and
- the probability scores for all four topics.

This ends up being the first step in the full news assistant pipeline.

```
import torch
from torch.nn.functional import softmax

id2label = {i: name for i, name in enumerate(label_names)}

def classify_text(text: str):
    enc = tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=128,
        return_tensors="pt",
    )
    # Move input tensors to the same device as the model
    enc = {k: v.to(model.device) for k, v in enc.items()}

    with torch.no_grad():
        outputs = model(**enc)
        probs = softmax(outputs.logits, dim=-1).squeeze().tolist()
    pred_id = int(torch.argmax(outputs.logits, dim=-1))
    return {
        "pred_label_id": pred_id,
        "pred_label_name": id2label[pred_id],
        "probs": {id2label[i]: float(p) for i, p in enumerate(probs)},
    }

# quick sanity check
classify_text("The Yankees beat the Red Sox in last night's game.")
```

```
{'pred_label_id': 1,
 'pred_label_name': 'Sports',
 'probs': {'World': 0.404343843460083,
  'Sports': 0.5897155404090881,
  'Business': 0.004883721936494112,
  'Sci/Tech': 0.0010568842990323901}}
```

8. Building the retrieval corpus and embeddings

For the RAG part of the project, I need a way to retrieve relevant news articles for a user’s question. To do this, I take a subset of AG News articles and turn them into vector embeddings.

In this step, I:

- select a portion of the dataset (about 10,000 articles),
- combine each article's title and description into one text string,
- and use a SentenceTransformer model to create dense embeddings for each article.

These embeddings are later added to a FAISS index so I can quickly search for the most similar articles during retrieval.

```
from sentence_transformers import SentenceTransformer
import numpy as np
import faiss

embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# Use a smaller subset for speed (e.g., 10k docs)
corpus_size = 10000
corpus = ag_news["train"].select(range(corpus_size))

corpus_texts = [
    corpus[i]["text"] for i in range(corpus_size)
]
corpus_labels = [corpus[i]["label"] for i in range(corpus_size)]

# Compute embeddings
corpus_embeddings = embed_model.encode(corpus_texts, batch_size=64, convert_to_
```

```
modules.json: 100%                                                    349/349 [00:00<00:00, 18.3kB/s]

config_sentence_transformers.json: 100%                               116/116 [00:00<00:00, 5.65kB/s]

README.md:         10.5k/? [00:00<00:00, 663kB/s]

sentence_bert_config.json: 100%                                       53.0/53.0 [00:00<00:00, 1.31kB/s]

config.json: 100%                                                     612/612 [00:00<00:00, 22.1kB/s]

model.safetensors: 100%                                               90.9M/90.9M [00:01<00:00, 162MB/s]

Loading weights: 100%  103/103 [00:00<00:00, 327.78it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                      | Status     |  |
-------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; r
tokenizer_config.json: 100%                                           350/350 [00:00<00:00, 10.4kB/s]

vocab.txt:         232k/? [00:00<00:00, 4.14MB/s]

tokenizer.json:         466k/? [00:00<00:00, 13.0MB/s]

special_tokens_map.json: 100%                                         112/112 [00:00<00:00, 2.87kB/s]

config.json: 100%                                                     190/190 [00:00<00:00, 5.95kB/s]
```

9. Creating a FAISS similarity index

After computing the embeddings, I normalize them and build a FAISS inner-product index. This lets me take a new query embedding and quickly find the most similar news articles in the dataset.

The FAISS index becomes the core retrieval system in the RAG pipeline.

```
# Build FAISS index (L2 with normalized vectors ~= cosine)
corpus_embeddings = corpus_embeddings.astype("float32")
faiss.normalize_L2(corpus_embeddings)
index = faiss.IndexFlatIP(corpus_embeddings.shape[1])
index.add(corpus_embeddings)

len(corpus_embeddings), index.ntotal
```

```
(10000, 10000)
```

10. Retrieval function

Here I define a `retrieve_docs` function that handles the search step. It:

1. Embeds the user's question with the same SentenceTransformer model,
2. Searches the FAISS index for similar articles,
3. Optionally filters the results to match the predicted topic,
4. And returns the top matching articles along with their text, label, and similarity score.

This gives the generator model the context it needs to produce an answer.

```
def retrieve_docs(query: str, k: int = 5, restrict_to_label: int | None = None)
    # embed query
    q_emb = embed_model.encode([query], convert_to_numpy=True).astype("float32"
    faiss.normalize_L2(q_emb)
    scores, idxs = index.search(q_emb, k * 3)  # search more, then filter

    idxs = idxs[0]
    scores = scores[0]

    retrieved = []
    for score, idx in zip(scores, idxs):
        label_id = corpus_labels[idx]
        if restrict_to_label is not None and label_id != restrict_to_label:
            continue
        retrieved.append(
            {
                "text": corpus_texts[idx],
                "label_id": label_id,
                "label_name": label_names[label_id],
                "score": float(score),
            }
        )
        if len(retrieved) >= k:
            break
    return retrieved

# quick test
retrieve_docs("latest developments in stock market", k=3)
```

```
[{'text': 'Stocks Up on Conflicting Economic Reports NEW YORK - Stocks inched
higher in quiet trading Wednesday as a pair of government reports offered
conflicting signals about the economy and oil prices declined.    Despite the
modest rise in share prices, investors have been in no hurry to commit new
money to stocks...',
  'label_id': 0,
  'label_name': 'World',
```

```
  'score': 0.6023176908493042},
 {'text': 'Stocks Are Up Despite Rising Oil Prices NEW YORK - Buyers put a
positive spin on equities Wednesday, shrugging off rising crude futures as
Google Inc. prepared to sell its stock in an initial public offering, albeit it
at a far lower price than previously forecast...',
  'label_id': 0,
  'label_name': 'World',
  'score': 0.5577706098556519},
 {'text': 'Stocks little changed on IBM, J amp;J news Stocks are little-changed
in early trading. The Dow Jones Industrial Average is down five points in today
#39;s early going. Losing issues on the New York Stock Exchange hold a narrow
lead over gainers.',
  'label_id': 2,
  'label_name': 'Business',
  'score': 0.5569276809692383}]
```

11. Answer generation with a pretrained seq2seq model

To turn the retrieved articles into a natural-language answer, I use a small pretrained seq2seq model called `flan-t5-small`.

The `generate_answer` function does three main things:

- it combines the retrieved documents into one context block,
- adds the user's question to that context,
- and uses Flan-T5 to generate a short answer based on that information.

This is the generation step of the RAG pipeline.

```
from transformers import AutoTokenizer as Seq2SeqTokenizer, AutoModelForSeq2Sec

gen_model_name = "google/flan-t5-small"
gen_tokenizer = Seq2SeqTokenizer.from_pretrained(gen_model_name)
gen_model = AutoModelForSeq2SeqLM.from_pretrained(gen_model_name)

def generate_answer(question: str, context_docs: list[dict], max_new_tokens: ir
    context = "\n\n".join([d["text"] for d in context_docs])
    prompt = (
        "You are a helpful news assistant.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {question}\n\n"
        "Answer concisely using the context above."
    )
    enc = gen_tokenizer(prompt, return_tensors="pt", truncation=True, max_lengt
    with torch.no_grad():
        out = gen_model.generate(
            **enc,
            max_new_tokens=max_new_tokens,
            do_sample=False,
```

```
        )
    return gen_tokenizer.decode(out[0], skip_special_tokens=True)
```

```
config.json:          1.40k/?  [00:00<00:00,  117kB/s]
tokenizer_config.json:          2.54k/?  [00:00<00:00,  164kB/s]
spiece.model:  100%                                                792k/792k  [00:00<00:00,  4.18MB/s]
tokenizer.json:          2.42M/?  [00:00<00:00,  51.3MB/s]
special_tokens_map.json:          2.20k/?  [00:00<00:00,  217kB/s]
model.safetensors:  100%                                           308M/308M  [00:07<00:00,  52.5MB/s]
Loading weights:  100%          190/190  [00:00<00:00,  658.91it/s,  Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weigh
generation_config.json:  100%                                       147/147  [00:00<00:00,  7.22kB/s]
```

12. End-to-end RAG pipeline

The `news_assistant` function puts all the pieces together. It:

1. Classifies the user's question into one of the four news topics using the finetuned BERT model,
2. Retrieves the most relevant articles from the FAISS index (filtered by the predicted topic),
3. And generates an answer using Flan-T5 with those retrieved articles as context.

The function returns the original question, the predicted topic, the topic probabilities, the retrieved documents, and the final generated answer.

This is the full news assistant and shows how finetuning and RAG can work together in one system.

```
def news_assistant(query: str, k: int = 5):
    # 1. Classify query into news topic
    cls = classify_text(query)
    topic_id = cls["pred_label_id"]
    topic_name = cls["pred_label_name"]

    # 2. Retrieve relevant docs, filtered by predicted topic
    docs = retrieve_docs(query, k=k, restrict_to_label=topic_id)

    # 3. Generate answer from context
    answer = generate_answer(query, docs)
```

```
    return {
        "query": query,
        "predicted_topic": topic_name,
        "topic_probs": cls["probs"],
        "retrieved_docs": docs,
        "answer": answer,
    }

# Example
result = news_assistant("What is the latest on the NBA game from last night?")
result["predicted_topic"], result["answer"]
```

```
('Sports',
 "LeBron James, Carmelo Anthony, and a couple of their Olympic teammates made
it over to watch last night's women's game between the United States and
China.")
```

13. Example query and qualitative results

At the end, I test the system with a few example questions. These include sports questions (like NBA games), business or finance questions, and science or tech questions.

For each example, I look at:

- the predicted topic,
- the generated answer,
- and one of the retrieved articles.

This helps me check whether the assistant is choosing reasonable topics, pulling helpful context, and giving answers that make sense.

```
result["retrieved_docs"][0]
```

```
{'text': 'New Jersey Nets Team Report - December 7 (Sports Network) - The New
Jersey Nets try for their third consecutive win this evening when they head to
Cleveland to face LeBron James and the Cavaliers at Gund Arena.',
 'label_id': 1,
 'label_name': 'Sports',
 'score': 0.5271512269973755}
```

Final Project Summary

Project Overview

For this project, I built a small news assistant that uses two techniques from our class: finetuning a pretrained model and using Retrieval-Augmented Generation (RAG). The idea is simple — the assistant takes a question about news, predicts which topic it belongs to (World, Sports, Business, or Sci/Tech), finds similar articles, and then generates a short answer based on those articles.

I used the AG News dataset for both parts of the project. It already includes thousands of labeled news articles, which makes it a good fit for training the classifier and for testing retrieval.

Methods

First, I finetuned the `bert-base-uncased` model as a 4-way topic classifier. I combined each article's title and description, tokenized it, split the data into training, validation, and test sets, and trained with the Hugging Face Trainer API.

Next, I built the RAG pipeline. I selected around 10,000 articles from the dataset and used the `all-MiniLM-L6-v2` SentenceTransformer model to create embeddings for each one. I added these embeddings to a FAISS index so I could quickly search for similar articles. When a user asks a question, the system predicts the topic using the finetuned BERT model, retrieves the top-k similar articles from that topic, and then uses `flan-t5-small` to generate an answer based on the retrieved text.

Results

On the AG News test set, the finetuned classifier achieved:

- **Accuracy:** ~0.94
- **Macro F1:** ~0.94

These results show strong performance across all four topics, especially considering the model was trained on only a fraction of the available data. When I tested the full system with example questions (mainly sports, business, and tech questions), it usually predicted the correct topic, retrieved relevant articles, and produced reasonable answers.

Discussion and Limitations